



Guía para conseguir el 100% en el "Hito 1"

El **Hito 1** es el cimiento sobre el cual construirán sus proyectos. El objetivo de esta etapa no es tener una aplicación web funcionando ni una base de datos conectada, sino **demostrar que dominan el modelado de negocio puro y las pruebas unitarias rigurosas**.

Para alcanzar el puntaje máximo (10/10 - Nivel "Excelente"), su repositorio debe cumplir estrictamente con tres pilares. Aquí les explicamos exactamente qué buscará el evaluador y cómo lograrlo.



Pilar 1: Core de Entidades de Dominio Puro (3 Puntos)

El objetivo: Crear un modelo de negocio que no sepa nada del mundo exterior (ni de bases de datos, ni de frameworks, ni de interfaces web).

Checklist para el Nivel "Excelente":

- **Cero Frameworks Mágicos:** Revisen todas las clases dentro de su paquete `domain` (o equivalente). **Está estrictamente prohibido** usar anotaciones de Spring (`@Service`, `@Autowired`), anotaciones de base de datos (`@Entity`, `@Table`) o dependencias web. Su core debe ser **Java Puro**.
- **English Only, Please:** La rúbrica exige código limpio y modular **en inglés**. Nombres de paquetes, clases, métodos, variables e incluso los mensajes de excepción internos deben estar en inglés. (Ej: `Event`, `Ticket`, `calculateTotal()`, `OutOfStockException`).
- **Inyección por Constructor:** Si un servicio de dominio necesita comunicarse con el exterior (ej. un repositorio de datos para guardar algo), debe depender de una **interfaz** (ej. `UserRepository`). Esa dependencia **debe** inyectarse obligatoriamente a través del constructor de la clase, sin usar frameworks.



Pilar 2: Suite Automatizada con JUnit 5 y Mockito (3 Puntos)

El objetivo: Demostrar que su código funciona y está protegido contra fallos mediante pruebas unitarias escritas con rigor profesional.

Checklist para el Nivel "Excelente":

- **Patrón AAA Estricto:** Sus tests no pueden ser un bloque de código desordenado. Deben estar divididos visual y lógicamente. **Les recomendamos usar comentarios para separar cada fase:**
 - `// Arrange` (Preparar): Instanciar objetos, crear datos ficticios, y configurar el comportamiento de los Mocks
`(when(mock.metodo()).thenReturn(valor)).`

- `// Act` (Actuar): La llamada a un (y solo un) método que están probando.
- `// Assert` (Afirmar): Las verificaciones (`assertEquals`, `assertTrue`) y la validación de interacciones con los mocks (`verify(mock).metodo()`).
- **Control de Excepciones de Negocio:** No basta con probar el "camino feliz". Si su sistema tiene reglas de negocio (ej. "No se puede transferir dinero sin saldo"), deben crear excepciones propias y probarlas usando `assertThrows`:

```
// Assert
assertThrows(InsufficientFundsException.class, () -> {
    // Act
    service.transfer(account, 1000);
});
```
- **Aislamiento Total:** El dominio no toca la base de datos real en este hito. Todas las dependencias externas (interfaces) deben ser simuladas usando **Mockito Core** (`@Mock`, `@InjectMocks` o instanciando los mocks manualmente).

100 Pilar 3: Cobertura Matemática del 100% (4 Puntos)

El objetivo: Respaldar con métricas que no existe una sola línea de lógica de negocio sin probar.

Checklist para el Nivel "Excelente":

- **Reporte en Verde (JaCoCo):** Deben configurar una herramienta de cobertura (como el plugin de JaCoCo en Maven/Gradle). Al generar el reporte HTML, el paquete de su dominio debe marcar **100%**.
- **Cero Caminos Huérfanos (Branch Coverage):** Este es el punto crítico. Si tienen una estructura condicional (`if/else` o `switch`), **deben** escribir un test que pase por cada rama posible. Si su reporte de JaCoCo muestra líneas amarillas, significa que les falta un test para cubrir una condición lógica.
- **Foco en el Core:** El 100% se exige sobre sus métodos de negocio críticos (el dominio). No es necesario en clases de configuración técnica o en DTOs si estos no contienen lógica condicional, pero el núcleo lógico no puede tener brechas.

El Entregable: Un `README.md` Profesional

Su repositorio en GitHub debe incluir un archivo `README.md` en la raíz que contenga:

1. **Título y Resumen:** Nombre de su proyecto y una línea explicando qué hace su dominio.
2. **Arquitectura:** Mencionar que implementan un Core de Entidades Puro (Clean Architecture / Puertos y Adaptadores).
3. **Comandos de Ejecución:** Los comandos exactos que el evaluador debe copiar y pegar en su terminal para comprobar su trabajo. Por ejemplo:
 - *Para correr la suite de tests:* `mvn clean test`
 - *Para generar el reporte de cobertura:* `mvn jacoco:report`

4. **Ruta de Evidencia:** Indiquen dónde abrir el reporte de cobertura generado (ej. <target/site/jacoco/index.html>).

¡Mucho éxito con el código! Céntrense en la calidad, el rigor y la pureza de sus clases antes de pensar en las siguientes etapas tecnológicas.